

SIMATS ENGINEERING
CO3 - ASSESSMENT TOOL - 2: Case Study

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Name	DHILIBAN M
Register Number	192421166

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Dave's Taxonomy: Precision (Level 3)
Question Count: 3

Total Marks: 30

Code of Conduct

I Dhiliban M /192421166 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Assessment Tasks - Case Study

1. Case Study 1: Smart Retail Inventory Management System

A retail chain implements a cloud-based inventory system connected to store devices. Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations.

2. Case Study 2: Cloud-Based Document Collaboration System

An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices. Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.

3. Case Study 3: Online Food Ordering Platform

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients. Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform. Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

Case Study Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application
Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis
Solution Design	20%	Solution is feasible, practical, and well justified	Reasonable solution with minor gaps	Basic solution	Unclear solution
Clarity	10%	Well-structured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized

Case Study 1: Smart Retail Inventory Management System

A retail chain implements a cloud-based inventory system connected to store devices. Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways.

Understanding of the case

A retail chain needs its point-of-sale terminals and management dashboards across many stores to stay synchronized with a centralized inventory system in real time, while also running analytics/forecasting on the accumulated data and keeping the whole system secure against external threats — all without requiring heavy on-site infrastructure at each store.

Mapping the architecture to cloud concepts

- Clients: POS systems (recording sales, updating stock) and management dashboards (viewing stock levels, reports) are the front-end clients.
- Web APIs: RESTful APIs sit between clients and cloud services, so every sale, restock, or stock-check is a discrete API call rather than a direct database connection — decoupling store hardware from backend implementation.
- Cloud storage: product catalog data, transaction logs, and analytics datasets are stored in a mix of a managed relational/NoSQL database (for live product and transaction records) and object storage (for bulk historical data used in analytics).
- Platform services: managed analytics and machine learning services (e.g., a managed data-warehouse plus a forecasting service) run demand forecasting without the retailer having to build and maintain ML infrastructure themselves.
- Security: VPNs connect store networks securely to the cloud, firewalls restrict traffic to only necessary ports/services, and an API gateway centralizes authentication, rate-limiting, and request validation for every client call.

Analysis and trade-offs

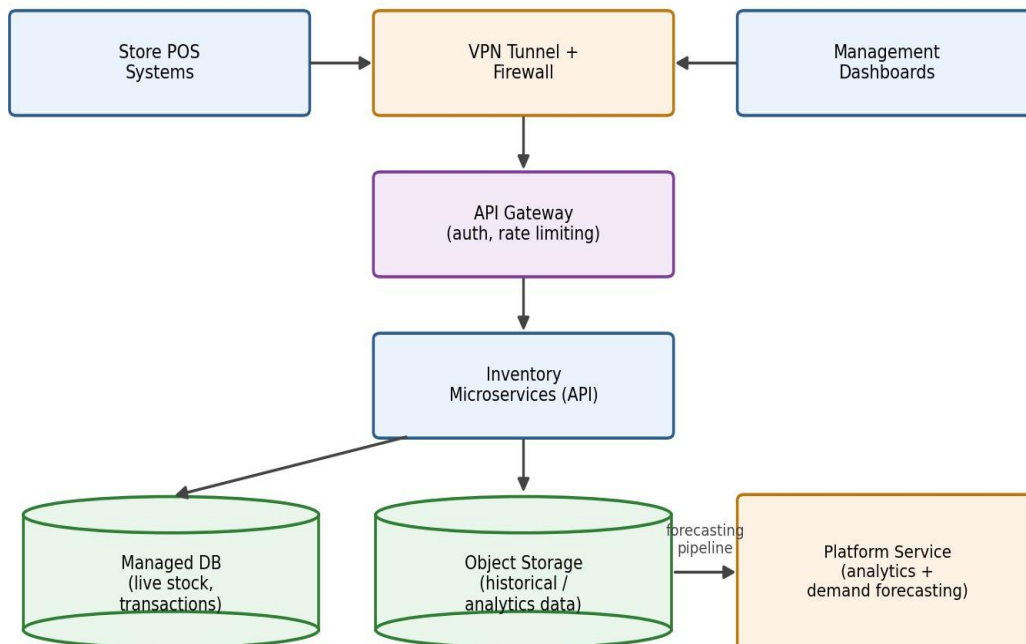
- Real-time sync vs. bandwidth/cost: syncing every transaction instantly gives dashboards up-to-the-second accuracy but increases API call volume and cost; batching updates every few minutes reduces cost but introduces staleness. A retail inventory system typically favors near-real-time sync for sales (to avoid overselling) but can batch less time-sensitive analytics data.
- Centralized vs. store-level processing: pushing all logic to the cloud simplifies management but makes stores dependent on connectivity; caching a local read-only copy of stock data at each POS terminal (with periodic reconciliation) improves resilience if the network briefly drops.

- Security layering trade-off: VPN + firewall + API gateway adds latency and operational complexity compared to a single perimeter control, but layered security is justified here because retail POS systems are a common target for payment-data breaches (PCI-DSS relevant), so defense-in-depth is worth the overhead.

Solution design

A store-side POS/dashboard client authenticates through an API gateway, which forwards validated requests over a VPN tunnel to cloud-hosted inventory microservices. Transactional data lands in a managed database for consistency, while a scheduled pipeline exports transaction and product data into object storage for the analytics/forecasting platform service to consume. Firewalls restrict inbound access to only the API gateway endpoint, and all store-cloud traffic is encrypted end-to-end.

Case 1 – Smart Retail Inventory Management Architecture



Case Study 2: Cloud-Based Document Collaboration System

An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices. Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.

Understanding of the case

The enterprise needs a browser-based, Google-Docs-style tool where multiple users can edit the same document simultaneously from anywhere in the world, see each other's changes in real time, keep a version history, and trust that their documents are secure and access-controlled.

Mapping the architecture to cloud concepts

- Clients: web browsers are the sole client — no local install — communicating over standard HTTPS/WebSocket connections.
- Web APIs: REST APIs handle document CRUD operations (create, open, share, list), while a persistent WebSocket-based API channel handles live, low-latency editing events between collaborators.
- Cloud storage: documents are stored in distributed cloud storage (object storage or a document-oriented database) with versioning enabled, so every edit can be tied to a specific version and rolled back if needed.
- Network infrastructure: a global CDN/edge network and multi-region deployment keep latency low for users regardless of location, which is critical for real-time collaborative typing to feel responsive.
- Security: encryption in transit (TLS) and at rest, an identity management system (SSO/OAuth) for login, and granular access permissions (owner, editor, viewer) enforced on every document.

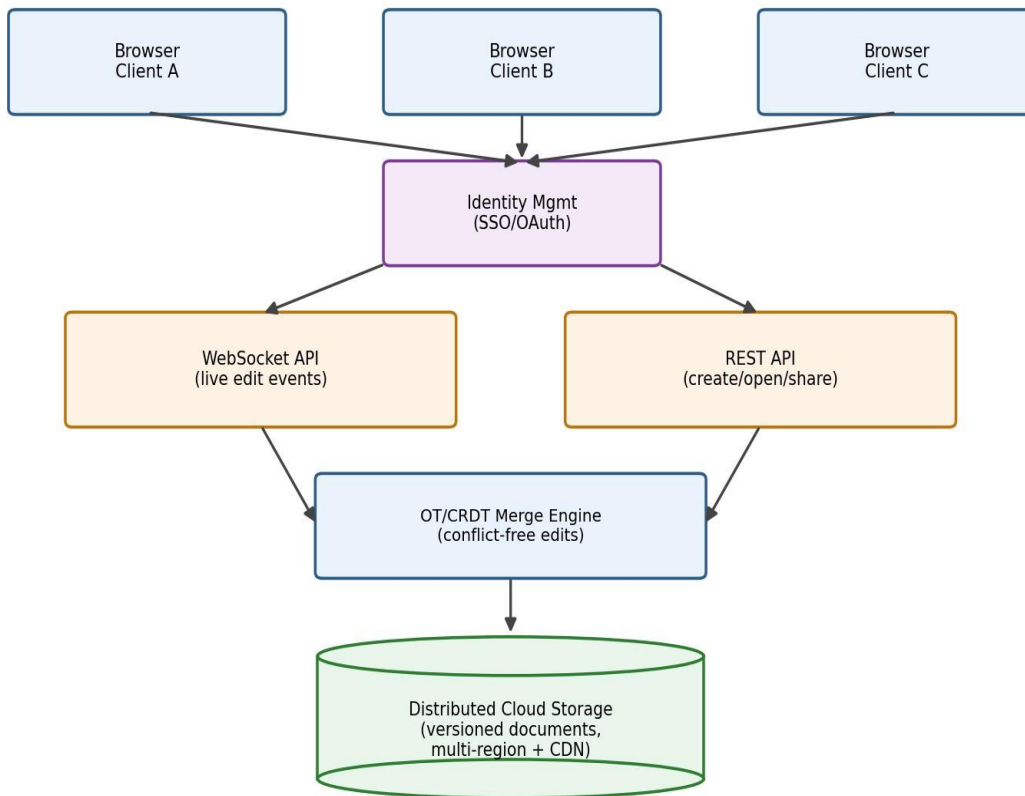
Analysis and trade-offs

- Consistency vs. availability (CAP-theorem trade-off): real-time collaborative editing across distributed users favors availability and low latency over strict consistency; systems typically use Operational Transformation (OT) or CRDTs (Conflict-free Replicated Data Types) to merge concurrent edits without locking the document, accepting eventual consistency in exchange for a smooth, non-blocking editing experience.
- Global low latency vs. data residency: replicating documents to edge locations worldwide improves speed but can conflict with data-residency requirements for enterprise/government customers — the design must allow document storage to be pinned to specific regions when required.
- Versioning granularity vs. storage cost: storing every keystroke as a version gives perfect audit history but costs more storage and processing; most systems snapshot versions at meaningful intervals (e.g., every few minutes or on explicit save) as a practical middle ground.

Solution design

Browser clients connect via a REST API for document management and a WebSocket channel for live edit events, both authenticated through the identity provider. Edits are merged using a CRDT/OT engine and persisted to versioned cloud storage, replicated across regions behind a CDN for low-latency global access. Access permissions are checked on every API and WebSocket request, and all data is encrypted both in transit and at rest.

Case 2 — Cloud Document Collaboration Architecture



Case Study 3: Online Food Ordering Platform

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients. Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform. Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced through authentication tokens, HTTPS, and role-based access control.

Understanding of the case

A startup needs a scalable, secure platform where customers order food through a website or mobile app, backend services process orders and payments reliably, and the system can grow with demand (e.g., lunch/dinner rush) without a large upfront infrastructure investment.

Mapping the architecture to cloud concepts

- Clients: web browser and mobile app clients both interact with the same backend, ensuring a consistent experience across platforms.
- Web APIs: RESTful APIs expose menu browsing, order placement, and order-status endpoints, decoupling the frontend from backend implementation details.
- Cloud storage: menu images are stored in object storage (optimized for serving static media, ideally via a CDN), while invoices and customer data are stored in a managed relational database for transactional integrity.

- **Infrastructure:** a mix of virtual machines (for steady-state backend services) and managed Kubernetes (for services needing elastic, on-demand scaling, such as order processing during peak hours) balances cost-efficiency with scalability.
- **Security:** authentication tokens (JWT/OAuth) validate every API call, HTTPS encrypts all client-server traffic, and role-based access control (RBAC) differentiates permissions between customers, restaurant partners, delivery staff, and administrators.

Analysis and trade-offs

- **VMs vs. managed Kubernetes:** VMs are simpler to operate for stable, predictable workloads (e.g., an admin backend), while Kubernetes better handles the food-ordering platform's variable, spiky load (lunch/dinner rushes) through auto-scaling — the hybrid approach lets the startup use the right tool for each workload rather than over-engineering everything onto Kubernetes or under-scaling everything on fixed VMs.
- **Cost vs. scalability:** managed Kubernetes has a steeper learning curve and slightly higher baseline cost than plain VMs, but for a startup expecting rapid growth, it avoids costly re-architecture later — a reasonable trade-off given the platform's growth trajectory.
- **Security overhead vs. usability:** enforcing HTTPS, tokens, and RBAC on every request adds a small latency/complexity cost but is essential given the platform handles payment and personal customer data; a lighter security model would be inappropriate here despite being "faster."

Solution design

Web and mobile clients call RESTful APIs (hosted behind a load balancer) for menu retrieval, order placement, and status tracking, all requests authenticated via short-lived tokens over HTTPS. Order-processing and matching services run on managed Kubernetes for elastic scaling during peak hours, while less-variable backend components (e.g., admin/reporting) run on standard VMs. Menu images are served from object storage via a CDN for fast load times, while invoices and customer records are stored in an encrypted, access-controlled relational database, with RBAC enforced at the API layer for every user role.

Case 3 — Online Food Ordering Platform Architecture

